Formalizing Computational Instrument Identity: The Geometry of the Forced Dependency Aperture

Driven By Dean A. Kulik

June 2026

1. Introduction: The Epistemological Crisis of Semantic Equivalence

For decades, the fields of computer science, compiler design, and formal software verification have struggled with a profound epistemological crisis regarding semantic equivalence. The fundamental challenge lies in answering a seemingly straightforward question: how can one definitively prove that two highly divergent sequences of code compute the exact same mathematical outcome without relying on exhaustive, infinite-bound input/output testing? Traditionally, the computer science community has approached this problem of program equivalence through the extraction of control-flow graphs (CFGs), input/output trace alignments, and the deployment of computationally expensive satisfiability (SAT) solvers.[1] However, these traditional methodologies are severely hindered by what is now formally recognized as the "carrier problem."

The carrier problem dictates that the physical and syntactical representation of an algorithm—whether it is written in high-level C source code, compiled down to target-specific assembly language, or represented as a flat hexadecimal binary—is fundamentally arbitrary.[3] Furthermore, identical source code processed by different compilers, such as the GNU Compiler Collection (GCC) and Clang, will result in drastically different execution sequences, register allocations, and transition streams.[3] These variations act as heavy, obscuring layers of dialect noise, burying the true mathematical intent of the algorithm beneath the idiosyncratic scheduling heuristics of the specific compiler.[5]

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828 – Ver Mark 9

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)
github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

This historic inability to separate the noise of the implementation from the core algorithmic geometry has severely limited fields ranging from superoptimization to malicious payload detection.[2] Obfuscation techniques easily manipulate control-flow graphs, rendering standard graph isomorphism checks practically useless due to their NP-intermediate computational complexity.[7] However, an exhaustive conceptual and empirical framework has recently dismantled this gridlock by proving that a real, formal, mathematically rigid object exists inside the noise of compilation. This is not a metaphor. The assertion that a cryptographic algorithm such as SHA-256 "feels like an instrument" has been successfully transformed into a reproducible, executable test.[3]

The core breakthrough establishes a definitive, load-bearing equation: the instrument identity is equal to its forced dependency aperture. The source code, the target assembly, the hexadecimal outputs, the compiler behaviors, the specific variable names, and the chronological execution orders are merely carriers. They are highly fluid and can change drastically. The aperture, conversely, is the invariant computational structure that strictly cannot change without irrevocably breaking the function itself.

This research report provides a comprehensive, exhaustive synthesis of the transition from abstract compiler stream analysis to the formal extraction of this dependency aperture. It thoroughly details the underlying theoretical frameworks, traces the empirical perturbation testing that verified the exact load-bearing geometries of the SHA-256 and Keccak algorithms, and examines the architecture of the Aperture Signature Compiler. Ultimately, this report demonstrates that we now possess a rigorous, automated compiler that identifies exactly what an executable thing is by discovering what cannot be perturbed without destroying its closure.

2. The Carrier Dialect and the Phase Ordering Problem

To truly understand the necessity of the forced dependency aperture, one must first deeply examine the mechanisms by which traditional compilers generate dialect noise. The modern optimizing compiler is a vastly complex pipeline that translates high-level imperative programming constructs into machine-executable instructions.[9] During this translation, compilers must simultaneously manage data dependencies, structural control flows, and hardware resource constraints. Two of the most critical and computationally demanding phases in this pipeline are instruction scheduling and register allocation.[4]

2.1 The Antagonism of Allocation and Scheduling

Instruction scheduling and register allocation are mutually antagonistic, NP-hard problems that create what is widely known in compiler theory as the "phase ordering problem".[4] Register allocation is the process of mapping a program's virtually infinite set of temporary variables to a finite, highly constrained set of physical hardware registers.[4] Instruction scheduling, conversely, involves reordering the execution sequence of those instructions to optimize for total latency, throughput, and the avoidance of dynamic hardware pipeline stalls.[4]

When a compiler prioritizes instruction scheduling (a pre-pass schedule), it tends to increase the number of concurrently active variables, which in turn spikes the register pressure.[5] High register pressure forces the

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828 – Ver Mark 9

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)
github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

register allocator to "spill" values to the stack memory, subsequently requiring additional load and store instructions that entirely alter the control-flow graph.[4] Conversely, if register allocation is prioritized, it restricts the freedom of the instruction scheduler, often creating artificial data dependencies that force the hardware to execute suboptimal, stalled sequences.[5]

Different compilers utilize drastically different algorithms to resolve this antagonism. Some rely on complex graph-coloring techniques to map interference graphs, while others utilize faster, greedy linear scan algorithms that simply scan instructions forward and spill registers dynamically based on immediate necessity.[5] Furthermore, modern compilers aggressively apply live range splitting, where a single variable is assigned to different physical locations at different stages of its lifespan, and coalescing, where copy instructions are actively eliminated by forcing variables to share the same hardware location.[4]

2.2 The Evolution of Intermediate Representations

The byproduct of these aggressive optimization heuristics is that the underlying algorithm is aggressively mutated. To mitigate this, compiler engineers have historically attempted to design sophisticated Intermediate Representations (IRs) that capture the essence of the program before the hardware-specific noise is applied. Early compilers relied heavily on the Control Flow Graph (CFG), which strictly modeled the chronological sequence of basic blocks.[13] However, CFGs overspecify the instruction and branch ordering, locking the mathematical intent into a rigid, artificial chronology.[14]

To introduce data flow awareness, static single-assignment (SSA) form and Continuation-Passing Style (CPS) were widely adopted, naturally fitting different paradigms of functional and imperative optimization.[14] Pushing this further, the Value Dependence Graph (VDG) and the Regionalized Value State Dependence Graph (RVSDG) were introduced to completely eliminate the CFG from the analysis phases.[9] The RVSDG is a purely data-flow-centric intermediate representation where nodes represent active computations, edges represent computational dependencies, and regions capture hierarchical structure, implicitly modeling entire programs in demand-dependence form without artificially dictating execution sequence.[9]

Yet, even with these advanced intermediate representations, proving whole-program semantic equivalence remained immensely difficult. Frameworks like the CompCert verified compiler achieve high assurance by meticulously proving that if a specific C program is successfully translated into assembly through its specific refinement passes, the source and the output are semantically equivalent.[2] However, this relies on the compiler verifying its own internal transitions. If an analyst is presented with two blind binary executables—one generated by GCC and one by Clang—and is tasked with proving they compute the identical algorithm, the internal refinement proofs are inaccessible.[2] The analyst is forced to rely on external graph isomorphism algorithms, which fail spectacularly when faced with structural deviations such as unrolled loops, reordered logic, or disparate temporary variable allocations.[6] The compiler's "carrier ceremony" permanently obscures the target.

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828 – Ver Mark 9

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)
github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

3. The Ontological Inversion and the Nexus Spine

The resolution to this epistemological gridlock requires a foundational paradigm shift, a theoretical realignment conceptualized extensively within the QuHarmonics research framework as the "Ontological Inversion".[16] Historically, standard scientific models rely implicitly upon a "Linear Stack" ontology, a hierarchical worldview that organizes existence in an upward-building methodology.[17] This worldview privileges "nouns"—static entities, persistent data structures, and predefined, immutable objects—over "verbs," which are the dynamic operational processes that actively generate state.[17]

The Ontological Inversion systematically dismantles this object-oriented, container-based approach to computational physics. It asserts rigorously that reality does not merely run on a passive computational substrate; rather, the substrate is fluidic, deterministic, and entirely self-executing—analogous to a Cosmic Field-Programmable Gate Array (FPGA).[16] Under this paradigm, an algorithm is not a passive data structure; it is an active, resonant geometry. The variable is merely the shape; the value is the fit, and computation is the act of carving that geometry into reality.[18]

3.1 The Hard Computational Spine

This theoretical framework transitions directly from philosophical intuition to hard computational machinery through the formalization of the Nexus computational spine. By treating the algorithm as an active geometry, the framework redefines the fundamental components of computation in structural, resonant terms.[3]

When the framework states that the output of an algorithm is a "note," it is defining the final readout value generated after the complete execution of the closure constraint.[3] The transition stream—the chronological sequence of CPU instructions executed by the machine—is merely a "performance." Like a piece of music, a performance can be played at different tempos, using different physical instruments, and with varied stylistic embellishments.[3]

The dependency cone is the "score," the actual structural graph of pathways that must be strictly traversed to achieve the note.[3] The perturbation test serves as the "load-bearing proof," a rigorous methodology of destructive boundary testing that definitively separates the necessary operational geometry from the arbitrary stylistic implementation.[3] Finally, the aperture hash serves as the "instrument identity." It is the immutable, mathematical signature that proves exactly what the computational object physically is, completely ignoring the language it was written in, the hardware platform it executes on, or the specific performance path it chose to take.[3]

This provides a grounded, rigorous mapping for intuition: the aperture is the set of dependencies that must mathematically close; the carrier is the stylistic method of traversal that can freely vary; and the forced aperture is the skeletal structure that strictly cannot be altered.[3]

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828 – Ver Mark 9

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)
github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

4. The Formal Geometry of Closure: The Aperture Tuple

To move beyond the limitations of Regionalized Value State Dependence Graphs and abstract syntax trees, the instrument identity must be reduced to its most foundational, inescapable mathematical constraints. This is achieved by formalizing the algorithm into an exact equivalence class governing forced dependency closure.[3]

A complex cryptographic hash function possesses an essentially infinite number of legal performances. It can be written as high-level C source code, translated into x86 assembly, represented as flat hex, generated as optimized GCC output, processed as Clang output, compiled with heavily unrolled execution loops, authored using vast arrays of explicit temporary-variable allocations, or calculated using alternate, mathematically equivalent logical formulas.[3] While all of these instantiations present completely different topological surfaces to a static analyzer, if they all preserve the exact same final computational closure, they are, by definition, the identical instrument.

This observation is formalized into the core mathematical equivalence theorem of the Aperture Signature schema.[3] Two distinct program execution paths, denoted as γ_a and γ_b, are mathematically equivalent if and only if they maintain the identical dependency closure under a specific formal grammar.[3]

$$\gamma_a \sim_{\Psi_f} \gamma_b \iff C(\gamma_a) = C(\gamma_b) = \Psi_f$$

Meaning: a different route that achieves the exact same closure equates to the exact same instrument. This is the executable realization of the Nexus premise—that there are many arbitrary paths, but only one forced closure.[3]

4.1 The Tuple Representation of the Instrument

To map this geometry computationally, the instrument f is stripped of its source code syntax and translated directly into a highly specific, directed acyclic tuple [3]:

$$\Psi_f = (V_f, E_f, A_f, C_f)$$

Each element of this tuple corresponds to a rigid, structural boundary that defines the limits of the instrument's operational geometry.[3]

The component V_f represents the formal forced dependency nodes. These are the specific, inescapable mathematical operations required to execute the instrument, entirely stripped of register allocations, target hardware restrictions, or variable naming conventions.[3] If a cryptographic function requires a non-linear bitwise AND gate to achieve diffusion, that specific operational node exists in V_f.[3]

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828 – Ver Mark 9

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)
github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

The component E_f represents the forced feed edges. This defines the required directional pathways of data propagation. It is the connective tissue of the tuple, dictating precisely how the output of one forced node must structurally feed into the input of another, entirely irrespective of how many temporary memory movements occur in the intermediary carrier execution.[3]

The component A_f represents the addressed carrier geometry. This defines the strict physical or logical topological layout where the data must reside.[3] Operations are not floating in a void; they are addressed to specific locations, such as the exact mathematical slots of an 8-word shift register, or the specific Cartesian coordinates of a multidimensional lane matrix.[3]

Finally, C_f represents the required closure condition, the strict reference behavior or expected test-vector output that definitively proves the network of nodes, edges, and addresses has correctly executed its required constraint.[3] By organizing the computation into this tuple, the theoretical framework effectively quarantines the noise. Any mutation applied to the source code that does not alter the V, E, A, C configuration is classified purely as carrier ceremony. Any mutation that disrupts any element of the tuple instantly destroys the mathematical closure.[3]

5. Discarding the Carrier Dialect: The Analytical Extraction Ladder

To empirically prove that the forced dependency tuple is a tangible geometric object rather than a convenient mathematical abstraction, researchers developed a multi-layered analytical extraction ladder.[3] This ladder was designed to systematically strip away the dialect noise of modern compilers, traversing downward from the highest level of execution chaos to the pristine, invariant core.[3]

The process was driven by Engine 23 and Engine 24, which were tasked with comparing the compiled outputs of the identical SHA-256 compression function source code processed by GCC 13.3 (at the O2 optimization level) and Clang 18.1 (at the O2 optimization level).[3]

5.1 The Transition Stream (Layers L0 to L3)

The analysis began at Layer L0, the raw, flat hexadecimal carrier body. At this layer, the divergence between the GCC and Clang executables was massive, with the $L1$ dialect noise distance frequently exceeding 0.50.[3] Moving one level deeper to Layer L2, researchers evaluated the basic operation histogram—effectively counting the "chords" or "notes" of the output without respecting execution order.[3] Engine 23 demonstrated that both GCC and Clang produce remarkably similar overall operational distributions, yielding an exceptionally low histogram $L1$ distance of 0.0507.[3]

However, the illusion of similarity collapsed at Layer L3, the transition stream layer. The transition stream tracks the "performance path"—the sequential, ordered motion of executing operations, mapping exactly how the program transitions from one semantic state to the next.[3]

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828 – Ver Mark 9

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)
github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

When analyzing the performance path, the strict architectural rhythms and proprietary heuristics of the differing compilers became starkly visible. Engine 23 extracted 161 individual instructions forming 160 transitions from the GCC output, and 138 instructions forming 137 transitions from the Clang output.[3] While some fundamental invariant core transitions remained relatively stable between the two—such as the stack -> control edge remaining identical at a delta of 0.000—the majority of the execution flow varied wildly.[3]

Semantic Transition Edge	GCC O2 Probability	Clang O2 Probability	Delta (Δ)
move -> rotate	0.039	0.159	0.120
rotate -> move	0.000	0.600	0.600
rotate -> rotate	0.300	0.000	0.300
logic -> logic	0.0375	0.0438	0.0063

(Table 1: Key dialect differences in transition stream probabilities between GCC and Clang, illustrating diverging architectural rhythms. Data sourced from Engine 23 transition metrics.[3])

The metrics revealed a profound structural variance. GCC heavily utilized an architectural rhythm sequencing its operations as rotate -> rotate -> logic, completely avoiding rotate -> move transitions. In stark contrast, Clang aggressively processed the same mathematical intent using a move -> rotate -> move rhythm, heavily prioritizing memory and register shuffling between rotational steps.[3] This resulted in a global transition-edge $L1$ distance of 0.2267 and a global transition-edge Jensen-Shannon Divergence (JSD) of 0.1157 bits.[3] The stream layer proved to be exceptionally strict; even if two compilers miraculously preserve identical operation counts, their sequential performance paths will wildly diverge.

5.2 The Dependency Cone (Layer L4)

To bypass these carrier-specific architectural rhythms, Engine 24 advanced to Layer L4, the extraction of the pure dependency cone.[3] By parsing the disassembled code into an explicit (mnemonic, written_regs, read_regs) format, researchers constructed a complete data-flow directed acyclic graph.[3]

Crucially, the graph was then rigorously filtered. All instructions related to stack management, memory movement, and control-flow branching were stripped away.[3] The graph was narrowed exclusively to the

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828 – Ver Mark 9

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)
github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

"SHA-logic" subgraph, where both endpoints of every extracted edge belonged strictly to the core mathematical subset of {rotate, shift, logic, add}.[3]

The results of this filtration were profound. When the carrier ceremony of register spilling and instruction scheduling was stripped away, the massive L3 divergence practically vanished. Both the GCC and Clang subgraphs collapsed down to exactly 68 instructions and exactly 24 pure SHA-logic edges.[3]

SHA-Logic Transition Edge	GCC O2 Distribution	Clang O2 Distribution	Delta (Δ)
rotate -> logic	0.2500	0.2500	**0.0000**
logic -> logic	0.3333	0.2500	0.0833
add -> add	0.2500	0.2917	0.0417
logic -> add	0.1667	0.2083	0.0416

(Table 2: The filtered SHA-Logic Dependency Cone distributions, displaying near-perfect alignment across essential load-bearing connections. Data sourced from Engine 24 metrics.[3])

Most significantly, the vital Σ-function signature chains (the rotate -> logic edges) were entirely identical between the two compilers, matching perfectly in count, weight, and displaying a delta of absolute zero.[3] The Jensen-Shannon Divergence plummeted from 0.1157 bits at the stream layer down to an astonishing **0.0071 bits** at the logic-cone layer.[3] This represents a sixteen-fold reduction in variance, equating to a mere 0.7% of the theoretical maximum divergence. The minimal remaining $L1$ variation (8.3%) was cleanly attributed to internal operation scheduling redistribution rather than any structural absence.[3] Engine 24 empirically proved that GCC and Clang preserve the exact same closure dependency geometry through vastly different execution streams.[3]

6. Proving the Load-Bearing Geometry: Engine 25 and SHA-256

Having successfully mapped the geometric shape of the dependency cone, the next necessary phase of the research was to empirically verify whether every constituent component of this shape was strictly load-

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828 – Ver Mark 9

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)
github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

bearing.[3] To achieve this, researchers developed Engine 25, an Aperture Closure Verifier designed to execute a comprehensive matrix of destructive structural perturbations against the SHA-256 algorithm.[3]

The exclusion condition hypothesis was clear: if the theoretical framework is sound, then mutating any forced dependency edge or node will instantly shatter the computational closure of the hash, whereas radically mutating the carrier ceremony will leave the closure perfectly preserved.[3]

6.1 The Topology of the SHA-256 Aperture

The SHA-256 algorithm operates under an ARX (Addition-Rotate-XOR) algebraic family framework, utilizing 32-bit word widths and an 8-word addressed shift register state geometry.[3] Engine 25 mapped the specific, inescapable topological feed structure required to execute a successful round.[3]

The forced topology demands that the precise addresses $e, f,$ and g must physically feed into the Choice ($Ch(e,f,g)$) logical gate.[3] The address e must independently feed into the $\Sigma_1(e)$ node, which strictly requires rotational constants of 6, 11, and 25.[3] The resulting outputs of the $\Sigma_1(e)$ node, the $Ch(e,f,g)$ gate, the address h, and the constant injection nodes $K[t]$ and $W[t]$ must rigorously converge and feed into a terminal $T1$ state.[3] Simultaneously, on the opposing side of the register, the addresses $a, b,$ and c must feed into the Majority ($Maj(a,b,c)$) logic gate, while address a feeds the $\Sigma_0(a)$ node, requiring rotational constants of 2, 13, and 22.[3] Both the $\Sigma_0(a)$ and the $Maj(a,b,c)$ operations must converge into the $T2$ terminal state.[3] Finally, the algorithm demands exact terminal slot updates, where the modular additions of $T1 + T2$ must rigidly route into the a' state slot, and the additions of $d + T1$ must route into the e' state slot.[3]

6.2 The Engine 25 Perturbation Matrix

Engine 25 systematically subjected this established geometric topology to three distinct classes of aggressive perturbations, aiming to validate the target reference vector ba7816bf....[3] The results yielded a perfect 18/18 execution score, unequivocally confirming the boundaries of the instrument's structural identity.[3]

The first class of tests, aimed at deliberately breaking the aperture, required the systemic dismantling of the structural nodes (V_f) and edges (E_f). Researchers methodically dropped the required rotr(e,6) operation entirely from the Σ_1 chain, removed the rotr(a,2) requirement from Σ_0, mutated critical XOR gates into OR gates within the logic chains, and entirely excised the load-bearing Ch, Maj, and constant injection

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828 – Ver Mark 9

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)
github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

operations.[3] As hypothesized, these node-removal and edge-perturbation tests resulted in a 9/9 failure rate. Modifying any required structural component instantly broke the SHA-256 closure, proving that every defined node is fundamentally load-bearing.[3]

The second class of tests evaluated the strict necessity of the addressed geometry (A_f). In these rerouting tests, the load-bearing operations were executed flawlessly at the mathematical level, but were intentionally addressed to incorrect physical or logical state register slots. For instance, researchers forced the execution of a mathematically perfect $\Sigma_0(b)$ sequence where the geometric coordinates demanded $\Sigma_0(a)$, executed $\Sigma_1(h)$ where $\Sigma_1(e)$ belonged, and physically swapped the coordinate positions of the Ch and Maj gates.[3] This class resulted in a 3/3 failure rate. The reroute tests critically proved that the abstract presence of an operation is insufficient; the algorithm demands that operations be bound to highly specific geometric coordinates to achieve mathematical closure.[3]

The third class of tests targeted the survival of the closure by radically mutating the carrier ceremony while keeping the underlying dependency tuple intact.[3] Researchers systematically replaced all ROTR n rotation instructions with their mathematically equivalent ROTL (32-n) counter-instructions.[3] They deployed alternate, highly convoluted logical formulas to calculate the Ch and Maj gates, such as executing the majority calculation via the alternate formula ((a^b)&(a^c))^a.[3] They reordered the commutative sequence of the $T1$ modulo additions, explicitly split the Maj calculation into sequential partial XOR operations, and injected arrays of explicit intermediate temporary variables into the execution path.[3] Despite these massive structural mutations to the source code layout, the execution passed with a perfect 6/6 survival rate.[3] The functional equivalence was maintained flawlessly because the forced dependencies were mathematically preserved, proving empirically that the dependency closure cannot vary without destroying the function.[3]

7. Cross-Instrument Invariance: The Keccak/SHA3 Proof

While the Engine 25 results successfully proved the existence of the SHA-256 aperture, it presented a severe vulnerability in the theoretical framework: was the extraction engine methodology merely a highly specialized, inadvertently hard-coded SHA-256 detector? To validate the strictly "instrument-agnostic" nature of the formalization, the precise extraction and perturbation matrix methodology was deployed against a fundamentally different, alien cryptographic architecture: Keccak (SHA3-256), facilitated by Engine 26.[3]

7.1 The Geometric Contrast of the Keccak Architecture

The underlying algebraic architecture of Keccak provides an extreme structural contrast to the ARX modular logic of SHA-256. Keccak possesses no shift registers, operates utilizing a 64-bit word width, and strictly

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828 – Ver Mark 9

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)
github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

utilizes a multi-dimensional 5×5 Cartesian lane matrix geometry.[3] Furthermore, Keccak's terminal operations possess absolutely no modular addition; its nonlinear properties are governed exclusively by bitwise XOR, permutation, and AND/NOT logic fields.[3]

Engine 26 successfully isolated the entirely different suite of geometric forced dependencies native to Keccak [3]:

- **The θ (Theta) Step:** A massive diffusion operation calculating column parity across all 5 spatial rows simultaneously, heavily reliant on targeted XOR logic ($D[x] = C[x-1] \oplus \mathrm{ROTL}(C[x+1], 1)$).[3]
- **The ρ (Rho) Step:** Responsible for executing specific, 64-bit lane-bounded rotations.[3]
- **The π (Pi) Step:** A strict geometric permutation step responsible for structurally re-indexing the spatial lanes via the coordinate mapping $B[y, 2x+3y] = \mathrm{ROT}(A[x,y], r[x,y])$.[3]
- **The χ (Chi) Step:** The singular source of non-linearity in the instrument, generating diffusion across adjacent rows via an explicitly defined AND/NOT bitwise gate ($A[x,y] = B[x,y] \oplus ((\neg B[x+1,y]) \wedge B[x+2,y])$).[3]
- **The ι (Iota) Step:** A targeted round constant injection fed exclusively into a single, specific geometric lane.[3]

7.2 The Coordinate Carrier Correction

During the initial deployment of the Engine 26 verification suite, the engine critically failed the reference assertion test before the perturbation suite could even commence.[3] Upon deep inspection, the failure was isolated not to a flaw in the theoretical tuple framework, but to a subtle, silent mismatch originating purely within the human-authored Keccak coordinate carrier.[3]

Specifically, the authoring script utilized an incorrect lane orientation array and a flawed π mapping formula.[3] Because of the extreme sensitivity of the geometric matrix, the incorrect orientation silently produced a plausible-looking but completely invalid mathematical digest.[3] Once the formal spatial constraints were applied—strictly enforcing the traditional index convention of $\mathrm{index} = x + 5y$ and strictly adhering to the precise π mapping of $B[y, 2x+3y]$—the reference implementation locked perfectly into alignment with the standard Python hashlib.sha3_256 digest.[3]

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828 – Ver Mark 9

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)
github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

This initial failure serendipitously reinforced the core thesis regarding the severity of the addressed geometry (A_f): the coordinate conventions are not immediately obvious from a high-level reading of the specifications, and executing the correct mathematical operations within an incorrectly mapped spatial geometry inherently breaks the computational closure.[3]

7.3 The Engine 26 Perturbation Matrix

Following the coordinate correction, the identical three-class perturbation methodology was executed against the Keccak geometry, ultimately yielding a perfect 16/16 execution score.[3]

In the aperture-breaking class, researchers successfully bypassed the θ step entirely, surgically removed the $\mathrm{rotl64}$ sub-operation from the θ parity calculation, skipped the execution of all ρ lane rotations, deleted the critical χ nonlinear step, replaced the vital AND logic inside the χ gate with an OR instruction, and skipped the ι round constant injection entirely.[3] As demanded by the theory, all 8 tests resulted in instantaneous closure failure.[3]

The reroute class proved equally sensitive. Dropping the geometric π step entirely (meaning the ρ lane rotations were mathematically performed, but simply lacked the structural spatial re-indexing) broke the closure immediately.[3] Similarly, shifting the column parity (θ) out of sequence to execute *after* the row nonlinearity (χ) violated the required dependency hierarchy, breaking closure and mirroring the failure seen when swapping Ch and Maj in SHA-256.[3]

Finally, the carrier ceremony class passed flawlessly. The carrier software representation was severely mutated by utilizing alternate rotl implementations, explicitly materializing the implicit NOT function within the χ gate via an XOR with MASK64 operation, unrolling the θ loops manually, injecting explicit temporary state variables into χ, and heavily reordering the commutative C XOR groupings.[3] All 5 tests passed seamlessly.[3]

This successful validation established the "L6 Result," a mathematical theorem confirming that different cryptographic instruments natively yield entirely different forced apertures.[3]

$$\Psi_{\mathrm{SHA}} \neq \Psi_{\mathrm{Keccak}}$$

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828 – Ver Mark 9

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)
github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

The diagnostic methodology successfully adapted to a completely different mathematical architecture without attempting to forcefully map Keccak into SHA structural terms.[3] This "Non-Collapse" principle proves conclusively that the extraction framework is truly instrument-agnostic; it dynamically reads the unique geometry of whatever active instrument is provided.[3]

8. Automating the Extraction: The Engine 27 Aperture Signature Compiler

The extraction of invariant geometries and the manual execution of perturbation tests through Engines 25 and 26 successfully established the theoretical foundation. However, to transition this framework from an isolated diagnostic testing suite into a robust, executable software verification ecosystem, Engine 27 was engineered to act as a fully automated Aperture Signature Compiler.[3]

Instead of deriving theoretical signatures from the output logs of previous engines, Engine 27 acts as a completely self-contained compiler pipeline: $f \longrightarrow \mathcal{A}(f)$.[3] It systematically ingests a cryptographic instrument's source specification, reference test vectors, and explicitly declared carrier-equivalence rules, processes them through a rigorous testing schema, and ultimately emits a formally serialized JSON signature object representing the canonical geometric aperture.[3]

8.1 The Two-Phase Sealed Compile Protocol

To ensure absolute cryptographic, mathematical, and logical integrity during the dynamic extraction process, the Engine 27 compiler architecture is governed by a strict two-phase "sealed compile" protocol.[3]

Phase 1 operates as the "Extract and Declare" cycle.[3] The compiler initializes by loading the instrument specifications and parsing the physical state geometry, determining whether the algorithm executes on a shift_register or a lane_matrix, and logging the required geometric width and exact slot/lane nomenclatures.[3] Simultaneously, the engine executes parallel extractions, pulling the initial candidate sets for the Nodes (V_0), Edges (E_0), Addresses (A_0), and Constants (C_0).[3]

Crucially, during this phase, the user is required to declare a specific parsing grammar (G_f, e.g., "sha256_round_v1" or "keccak_round_v1") alongside an exhaustive structural perturbation policy spanning node tests, edge tests, address tests, constant tests, and ceremony tests.[3]

Once this data is extracted and declared, Phase 1 terminates abruptly at a hard, cryptographic "Commit Barrier".[3] At this barrier, the compiler's initial state is completely frozen, and a commit_hash is permanently generated:

$$\mathbf{K_0} = \mathrm{Hash}(\mathrm{canon}(V_0, E_0, A_0, C_0, G_f))$$

The commit_hash serves as absolute proof that the run was securely sealed *before* any verification processes commenced.[3] Once sealed by $\mathbf{K_0}$, the candidate sets are permanently locked, the grammar G_f is frozen,

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828 – Ver Mark 9

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)
github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

and absolutely no new rules, structural nodes, or routing edges may be injected mid-run.[3] Consequently, the mathematical dimensions of the coverage ledger matrix—consisting of rows (q) for each extracted theoretical candidate and columns (p) for each generated perturbation test defined in the policy—are completely fixed.[3]

Phase 2 then commences as a highly restricted "Closed Loop" verification cycle.[3] The compiler systematically generates the vast suite of perturbations across the fixed sets, executing each perturbed model (γ') against the reference instrument to log whether the output constraint state survives ($C(\gamma') = \Psi$) or fails ($C(\gamma') \neq \Psi$).[3] The results of every test are methodically mapped into the rigid $M[q, p]$ matrix ledger.[3]

8.2 Survival Contracts and the Coverage Ledger

Step 9 of the Engine 27 protocol serves as the ultimate gatekeeper, governing the type-specific survival contracts that determine whether a candidate q is genuinely load-bearing.[3] For a candidate to survive the ledger and be officially permanently inscribed into the final instrument identity (Ψ_f), it must adhere strictly to behavioral conditions.[3]

Operational Nodes (V) must explicitly break the closure when removed, but must flawlessly pass the closure when subjected to ceremony rewrites.[3] Edges (E) face a harsher requirement: they must break the closure when removed, break the closure when rerouted to incorrect coordinates, and yet pass cleanly under ceremony.[3] The state Addresses (A) must similarly break closure when rerouted, but pass under ceremony variations.[3]

Conversely, elements that exist purely for software optimization—such as the explicit temporary variable t1 frequently utilized in C carriers—will safely pass every single ceremony test without breaking the closure.[3] Consequently, the ledger classifies these elements strictly as non-load-bearing "ceremony only" objects, ruthlessly excluding them from the final topological signature.[3]

Candidate (q)	Break Req.	Reroute Req.	Ceremony Preserve	Ledger Verdict (→Ψf)
$\Sigma_1(e)$ **Node**	breaks	breaks	passes	**Load-bearing inclusion**

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828 – Ver Mark 9

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)
github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

$Ch(e, f, g)$ **Gate**	breaks	breaks	passes	**Load-bearing inclusion**
π **Reindexing**	breaks	breaks	passes	**Load-bearing inclusion**
Temp Variable t1	n/a	n/a	passes	**Excluded (Ceremony only)**

(Table 3: A synthesized representation of the Step 9 Coverage Ledger, demonstrating the filtering mechanism based on strict survival contracts.[3])

8.3 The Omega (Ω) Trigger Constraint

A hallmark of a mathematically rigorous formal system is how it aggressively manages internal errors, unexpected computational anomalies, or missing architectural data. Because the matrix ledger is cryptographically sealed by the commit_hash prior to execution, a critical vulnerability exists: the declared grammar could have inadvertently missed a hidden but mathematically essential operational dependency during the open Phase 1 extraction.[3]

The Engine 27 protocol manages this potential failure cascade through the implementation of the Ω (Omega) trigger constraint.[3] If, during the closed-loop perturbation testing, the survival contract of a known, tracked candidate q unexpectedly relies upon referencing an external dependency q' that possesses no corresponding row in the frozen ledger, a "True Ω Condition" is instantly triggered.[3]

In traditional heuristic compilers, the system might attempt to dynamically patch itself, silently retrofitting the missing variable into the matrix to avoid a compilation failure.[5] The Aperture Compiler is strictly forbidden from doing so. The Ω trigger instantly aborts the entire execution loop, logs a severe exception (Ω_CANDIDATE_DISCOVERY_AFTER_COMMIT), increments the grammar version designation (e.g., from v1 to v2), and forces a complete, hard restart of Phase 1.[3] This draconian constraint guarantees that the ledger cannot be dynamically manipulated or padded mid-run to achieve a passing score, ensuring total systemic integrity.

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828 – Ver Mark 9

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)
github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

8.4 Canonicalization and the Aperture Signature Hash

If the execution loop completes perfectly without triggering an Ω exception, and all surviving candidates are successfully evaluated against their rigorous survival contracts, the compiler initiates Step 10: Canonicalization.[3] The final surviving geometric structure is aggressively normalized. Nodes are formally sorted by role, operation, parameters, and address coordinates; edges are sorted strictly lexicographically.[3] Operational equivalents are mathematically normalized against established substitution rules—for example, automatically transforming all detected $\mathrm{ROTR}(n)$ instances directly into canonical $\mathrm{ROTL}(w-n)$ equivalents, or standardizing alternate XOR forms of the θ parity calculation into a single canonical presentation.[3]

Finally, the canonicalized text serialization of these surviving elements—comprising the state_geometry, the load-bearing nodes (V_f), the pathways (E_f), the geometry (A_f), the closure (C_f), and the normalization rules—is processed through a SHA-256 algorithm.[3]

This yields the ultimate computational artifact, the **Aperture Hash**:

$$\mathcal{A}_G(f) = Hash(\mathrm{canon}_G(V, E, A, C))$$

This hash contains absolutely zero traces of the original author's source code.[3] It contains no traces of the target assembly language, no outputs of the test-vectors, no compiler-specific scheduling heuristics, and actively excludes all defined carrier-rewrite procedures (such as loop unrolling or commutative grouping rules).[3] It is purely a cryptographic fingerprint of the instrument's forced dependency aperture.[3]

9. Defining the Theoretical Boundaries: What the Aperture is Not

While the extraction of the geometric aperture represents a paradigm shift in semantic verification, it is critical to explicitly bound the theoretical implications of this framework to prevent misinterpretation.

The successful isolation of the SHA-256 and Keccak forced dependency geometries does not mean that the SHA-256 algorithm is cryptographically broken. It does not imply that standard cryptographic hashes are mathematically reversible, nor does it suggest that a final output digest contains the original source message encoded within it in a recoverable form.

The breakthrough is strictly structural. It means that the computer science community now possesses a mathematically rigorous, automatable mechanism to definitively tell the difference between the surface costume of an algorithm (the carrier dialect, the implementation choices, the compiler behavior) and the actual, load-bearing instrument identity. Hexadecimal strings, raw assembly language, C code, Python scripts, mathematical notations, and abstract diagrams are merely carriers; they are simply different, highly variable ways of holding the exact same executable shape. The real identity is exclusively defined as what cannot be changed without breaking the computational closure.

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828 – Ver Mark 9

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)
github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

10. Broader Implications for Formal Verification and Analysis

The transition from abstract compiler analysis to an executable aperture signature forces a severe recalibration of multiple sub-disciplines within computer science, spanning from formal verification methodologies to advanced cybersecurity analysis.

In fields involving software watermarking, plagiarism detection, and malicious payload identification, analysts routinely rely on subgraph isomorphism matching, semantic differencing, and abstract syntax tree comparisons to track algorithms.[6] These methods are highly fragile and computationally expensive. Because graph isomorphism is an NP-intermediate problem, attackers can effortlessly inject obfuscation noise—artificially unrolling execution loops, scrambling register allocations, inserting null operations, or heavily shuffling basic block layouts.[7] This effectively mutates the surface layer enough to easily evade traditional detection while retaining the underlying malicious execution logic.[6]

By fundamentally transitioning to a forced dependency aperture model, these widespread obfuscation techniques are rendered mathematically inert. Because the Engine 27 Aperture Signature Compiler explicitly filters out "ceremony only" components and maps solely the load-bearing geometric paths that dictate mathematical closure, the injected noise is completely bypassed.[3] An aggressively obfuscated cryptographic implementation will yield the exact same $A_G(f)$ Aperture Hash as the pristine, standard reference model.[3] This collapses the highly complex graph isomorphism problem into a definitive, linear cryptographic signature comparison, allowing security analysts to rapidly identify algorithms regardless of how heavily their source code has been mutated or hidden.

Furthermore, this framework completely bypasses the traditional limitations of whole-program semantic equivalence. Historically, verifying that an aggressively optimized compiler pass did not alter the fundamental execution logic required falling back on randomized input testing or bounded model checking, hoping that an unmapped execution state wouldn't cause a failure.[1] The aperture theorem eliminates this uncertainty by abstracting out the CPU state management entirely.[2] Instead of attempting to laboriously prove that Code X and Code Y modify the physical CPU hardware state in identical manners across infinite variables, the methodology proves that Code X and Code Y project the identical topological dependency network.[3] Because the final closure condition C_f mathematically forces all execution paths into the exact same structural convergence, whole-program semantic equivalence is guaranteed inherently, rapidly verifiable through a simple hash string alignment.[3]

11. Conclusion: A Formal Object Inside the Noise

The prevailing methodologies of computational analysis, compiler optimization, and formal verification have operated for decades under the assumption that a program's identity is inextricably bound to its source representation, its intermediate control-flow maps, or its final target-specific machine code layout. This inherent reliance on the arbitrary surface text of algorithms has resulted in a landscape where identifying

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828 – Ver Mark 9

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)
github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

equivalent logic across disparate architectures requires expensive, highly fragile heuristic-driven pattern matching, susceptible to the slightest variations in register allocation and instruction scheduling.

The comprehensive suite of extraction testing engines developed under the QuHarmonics framework—scaling from the extraction of refined dependency cones in Engine 24 [3], progressing through the rigorous, destructive perturbation exclusions of Engine 25 and Engine 26 [3], and ultimately culminating in the cryptographically sealed compilation architecture of Engine 27 [3]—demonstrates definitively that this assumption is fundamentally flawed.

By applying the rigorous standard that changing carrier ceremony must preserve closure while mutating a forced dependency must inherently break closure, researchers have successfully isolated a true formal mathematical object from the overwhelming chaos of modern compiler execution. The structural signatures of complex mathematical functions like SHA-256 and Keccak/SHA3-256 are not mere conceptual abstractions or localized behavioral descriptions; they possess a distinct, rigidly defined, and objectively verifiable physical geometry.[3]

The automation of this extraction process through the **Aperture Hash** ($A_G(f)$) artifact represents a massive, foundational breakthrough for computer science.[3] It empowers computational systems to instantly verify the true structural intent of any algorithm by deeply inspecting its invariant, load-bearing architecture, completely discarding the arbitrary stylistic implementation choices of the human developer or the proprietary scheduling heuristics of the host compiler. In doing so, it fulfills the deepest, most rigorous technical requirement of the theoretical Nexus premise: building a compiler that identifies exactly what an executable thing is, solely by finding the precise set of dependencies that cannot be perturbed without destroying its closure entirely.

Works cited

1. Semantic Program Alignment for Equivalence Checking - Stanford CS Theory, accessed June 2, 2026, https://theory.stanford.edu/~aiken/publications/papers/pldi19.pdf
2. Proving equivalence of programs - Stack Overflow, accessed June 2, 2026, https://stackoverflow.com/questions/44703441/proving-equivalence-of-programs
3. Keccak-aperture-correction-and-cross-instrument-analysis.md
4. Combinatorial Register Allocation and Instruction Scheduling - arXiv, accessed June 2, 2026, https://arxiv.org/pdf/1804.02452
5. Cooperative instruction scheduling with linear scan register allocation - NUS Computing, accessed June 2, 2026, https://www.comp.nus.edu.sg/~wongwf/papers/hipc05.pdf
6. Subgraph Isomorphism Based Intrinsic Function Reduction in Decompilation - SCIRP, accessed June 2, 2026, https://www.scirp.org/journal/paperinformation?paperid=64869

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828 – Ver Mark 9

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)
github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

7. Graph isomorphism - Wikipedia, accessed June 2, 2026, https://en.wikipedia.org/wiki/Graph_isomorphism
8. Subgraph Isomorphism Based Intrinsic Function Reduction in Decompilation - Semantic Scholar, accessed June 2, 2026, https://pdfs.semanticscholar.org/75de/5492f9877d3dd3803c9263d14551bfc354a2.pdf
9. RVSDG: An Intermediate Representation for Optimizing Compilers, accessed June 2, 2026, https://www.sjalander.com/research/pdf/sjalander-tecs2020.pdf
10. Combining Register Allocation and Instruction Scheduling (Technical Summary) - Computer Science, accessed June 2, 2026, https://cs.nyu.edu/media/publications/TR1995-698.pdf
11. CS415 Compilers Instruction Scheduling - CS-Rutgers University, accessed June 2, 2026, https://www.cs.rutgers.edu/courses/415/classes/spring_2016_zhang/lectures/lec04.pdf
12. Register allocation and spilling, the easy way? - Stack Overflow, accessed June 2, 2026, https://stackoverflow.com/questions/1960888/register-allocation-and-spilling-the-easy-way
13. Value Dependence Graphs: Representation Without Taxation Abstract 1 Introduction, accessed June 2, 2026, https://homes.cs.washington.edu/~mernst/pubs/vdg-popl94.pdf
14. Optimizing compilation with the Value State Dependence Graph - Department of Computer Science and Technology | - University of Cambridge, accessed June 2, 2026, https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-705.pdf
15. What I talk about when I talk about IRs | Max Bernstein, accessed June 2, 2026, https://bernsteinbear.com/blog/irs/
16. (PDF) The Nexus RHF: A Meta- Computational Synthesis of Topology, Cryptography, and the Geometry of Residue - ResearchGate, accessed June 2, 2026, https://www.researchgate.net/publication/405200951_The_Nexus_RHF_A_Meta-_Computational_Synthesis_of_Topology_Cryptography_and_the_Geometry_of_Residue
17. A Grand Unified Mathematical Ontology of Recursive Harmonic Operations Across Diverse Domains - Zenodo, accessed June 2, 2026, https://zenodo.org/records/18994837/files/A%20Grand%20Unified%20Mathematical%20Ontology%20of%20Recursive%20Harmonic%20Operations%20Across%20Diverse%20Domains.pdf?download=1

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828 – Ver Mark 9

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)
github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

18. The Nexus Harmonic Universe - The Ontological Inversion of the Variable. - ResearchGate, accessed June 2, 2026, https://www.researchgate.net/publication/402962125_The_Nexus_Harmonic_Universe_-_The_Ontological_Inversion_of_the_Variable
19. Program Slicing, accessed June 2, 2026, https://people.eecs.ku.edu/~saiedian/Teaching/814/Readings/intro-program-slicing.pdf

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828 – Ver Mark 9

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)
github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com